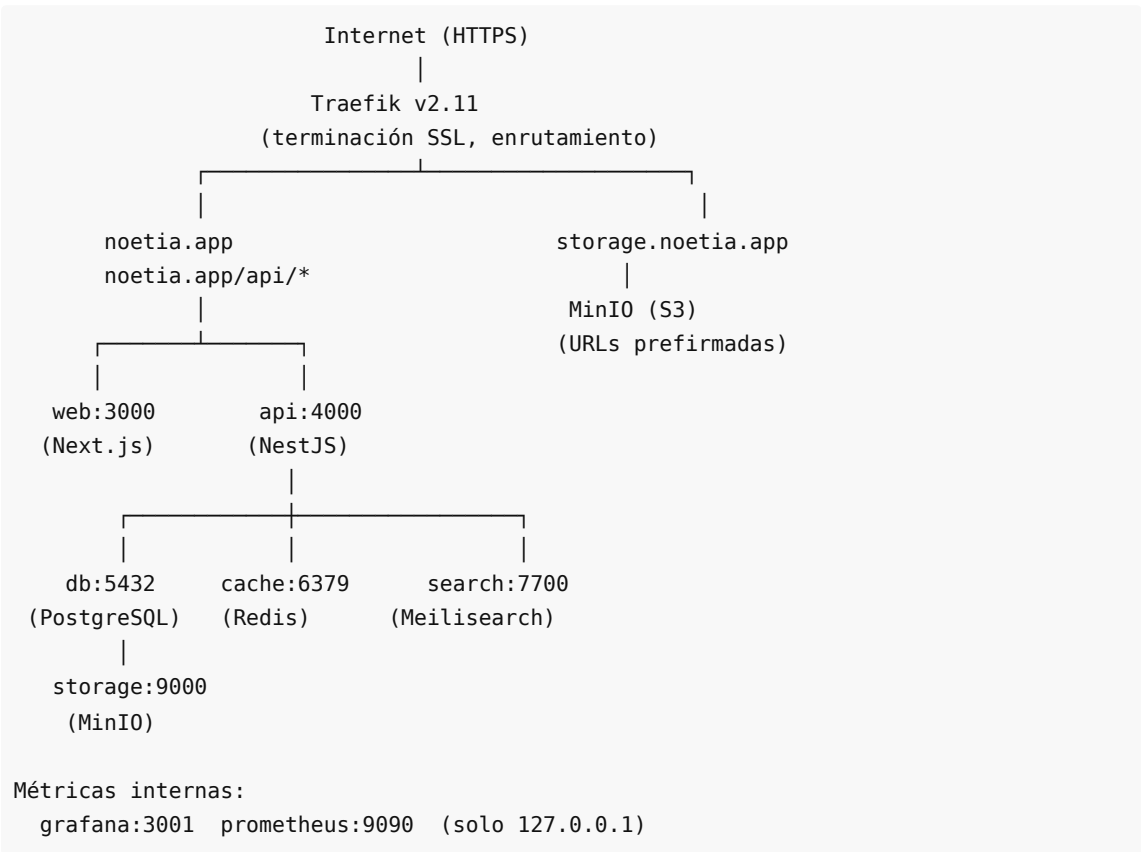


Noetia — Documento de Arquitectura Técnica

Versión 1.0 | Mayo 2026

1. Visión General del Sistema

Noetia es una plataforma de lectura multimodal compuesta por 9 servicios Docker, coordinados a través de un monorepo y desplegados en un único VPS detrás de un reverse proxy Traefik. El sistema está diseñado para un equipo pequeño con una huella operacional reducida — todos los servicios corren en una sola máquina con dependencias externas mínimas.



Inventario de Servicios

Servicio	Tecnología	Propósito
api	NestJS (Node.js + TypeScript)	API REST, lógica de negocio, autenticación, suscripciones
web	Next.js 14 (React)	Lector web, biblioteca, compartir, perfil, admin
db	PostgreSQL 16	Almacén de datos principal
cache	Redis 7	Sesiones, almacén de tokens, colas de jobs BullMQ
storage	MinIO (compatible con S3)	Libros, archivos de audio, imágenes generadas

search	Meilisearch v1.7	Búsqueda full-text de libros y fragmentos
worker	BullMQ (Node.js)	Jobs asíncronos: renderizado de imágenes, exportaciones de compartir
image-gen	Python 3 + Flask + Pillow	Microservicio de generación de imágenes de tarjetas de citas
proxy	Traefik v2.11	Reverse proxy, SSL, enrutamiento (solo en producción)
monitor	Grafana + Prometheus	Métricas, alertas, dashboards

2. Stack Tecnológico

Backend — API NestJS

Por qué NestJS: NestJS ofrece un framework estructurado y opinionado para Node.js que impone separación de responsabilidades mediante módulos, controladores, servicios y guards. Esto facilitó que un solo desarrollador backend construyera una API grande (más de 85 endpoints) con patrones consistentes. El sistema de inyección de dependencias permite pruebas unitarias limpias — cada servicio puede probarse con dependencias simuladas.

Patrones arquitectónicos clave:

- **Módulo por dominio:** Cada área de producto (auth, books, clubs, subscriptions, stats, etc.) es un módulo NestJS autónomo
- **JwtAuthGuard en todas las rutas protegidas:** Aplicado a nivel de controlador, no por endpoint
- **ThrottlerModule:** Rate limiting global de 120 req/min con sobrescrituras por ruta para endpoints de autenticación
- **TypeORM con migraciones:** Cero migraciones SQL manuales; cada cambio de esquema pasa por un archivo de migración versionado
- **SQL puro para consultas críticas de rendimiento:** Las estadísticas agregadas, cálculos de racha y UPSERTs de resolución de conflictos usan `repo.query()` directamente

Autenticación:

- Local (email + contraseña) con hash bcrypt
- Tokens de acceso JWT (vencimiento 15 minutos) + tokens de refresh (almacenados en Redis, TTL 7 días)
- Google OAuth, Facebook OAuth, Apple Sign-In — todos normalizados a la misma entidad User
- Tokens de confirmación de email y de reseteo de contraseña — firmados con HMAC, vida corta, almacenados en Redis

Frontend — Web Next.js

Por qué Next.js: Renderizado del lado del servidor para SEO en páginas públicas (landing, precios, descubrimiento de biblioteca), hidratación del lado del cliente para el lector interactivo. El App Router (Next.js 14) permite code splitting a nivel de ruta, manteniendo el bundle de la página del lector por debajo de 120 kB de First Load JS.

Decisiones de arquitectura:

- **Componentes 'use client' solo donde se necesitan** — las páginas de fetching de datos usan componentes de servidor donde sea posible
- **Utilidad `apiFetch`** : Un wrapper ligero sobre `fetch` que inyecta el JWT desde `localStorage` y lanza excepciones en respuestas no-2xx. Mantiene todas las llamadas a la API consistentes.
- **i18n personalizado (sin librería externa)**: Un contexto `LanguageProvider` con hook `useTranslation()`, objetos de traducción tipados para español e inglés, y sincronización vía API al montar. Se eligió sobre `next-i18next` para evitar complejidad en el build.
- **Sin gestor de estado global**: Contexto React para autenticación e idioma; `useState` local para el estado a nivel de página. La app no es lo suficientemente compleja para justificar Redux o Zustand.

Móvil — React Native (Expo SDK 51)

Por qué Expo (managed workflow): El flujo de trabajo gestionado de Expo significa que los directorios nativos `android/` e `ios/` no se commitean — EAS Build los regenera vía `expo prebuild` en el momento del build. Esto reduce dramáticamente la sobrecarga de mantenimiento nativo para un solo desarrollador frontend que gestiona ambas plataformas.

Arquitectura móvil clave:

- `expo-av` para reproducción de audio con modo de fondo (`UIBackgroundModes: audio`)
- `expo-notifications` para push (APNs en iOS, FCM en Android vía `google-services.json`)
- `expo-apple-authentication` para Sign in with Apple nativo
- `AsyncStorage` para datos offline (frases descargadas del libro, progreso de lectura)
- `NetInfo` para detección offline y sincronización al reconectar
- `EAS Update` para actualizaciones JavaScript OTA entre builds de tienda (canal: producción)

Base de Datos — PostgreSQL 16

Principios de diseño del esquema:

- Todas las claves primarias son UUIDs (`uuid_generate_v4()`)
- Todas las claves foráneas tienen `ON DELETE CASCADE` salvo que los datos deban preservarse tras la eliminación del padre
- Cada consulta multi-tenant filtra primero por `userId` — no es posible la fuga de datos entre usuarios
- Timestamps: `createdAt` (`CreateDateColumn`) y `updatedAt` (`UpdateDateColumn`) en todas las entidades principales
- Eliminaciones suaves vía `deletedAt` en mensajes de club (requisito de moderación de contenido)

Índices de rendimiento (migración 032):

- `idx_books_published_free` — cubre la consulta de descubrimiento de biblioteca
- `idx_books_collection` — cubre las consultas de agrupación de colecciones
- `idx_books_category` — cubre las consultas de filtro por categoría
- `idx_subscriptions_plan` — cubre la búsqueda de plan en el procesamiento de webhooks
- `IDX_reading_stats_user_date` — cubre el UPSERT de heartbeat de 7 días

Tablas principales:

Tabla	Estimación de filas (Año 1)	Notas

users	5,000	Identidad principal, configuración de privacidad, metas, uiLanguage
books	150	38 gratuitos + catálogo de autores en crecimiento
sync_maps	150	~1 por libro; array de frases almacenado como JSONB
reading_progress	15,000	1 fila por (usuario, libro)
reading_stats	365,000	1 fila por (usuario, fecha) — UPSERT diario
fragments	50,000	Resaltados de usuario
subscriptions	5,000	1 por usuario
token_ledger	15,000	Log de redención de tokens FIFO
clubs	500	Clubes públicos + privados
club_members	5,000	Muchos-a-muchos usuarios ↔ clubes
club_messages	100,000	Chat general del club
club_discussions	50,000	Comentarios anclados a frases

3. Infraestructura y Despliegue

Servidor de Producción

Especificación	Valor
Proveedor	Contabo Cloud VPS 30 SSD
CPU	8 vCPU
RAM	24 GB
Almacenamiento	400 GB SSD
Red	600 Mbit/s, ilimitado
SO	Ubuntu 24.04 LTS
Ubicación	Europa (Alemania)

Por qué Contabo: Óptimo en costo para la fase bootstrapped. 8 vCPU y 24 GB de RAM soporta 9 servicios Docker cómodamente, con capacidad para escalar a ~10,000 usuarios activos antes de necesitar un upgrade de VPS. En ese punto, migrar a PostgreSQL gestionado (RDS/Supabase) y un VPS más grande es el camino natural.

Estrategia de Docker Compose

Tres archivos compose separan las preocupaciones:

Archivo	Usado para
---------	------------

docker-compose.yml	Desarrollo local (montajes de volumen para hot reload, Mailhog para email)
docker-compose.prod.yml	Overlay de límites de recursos para producción
docker-compose.server.yml	Despliegue en producción (etiquetas Traefik, sin nginx, sin Mailhog)

Estrategia de montaje de volúmenes (solo en desarrollo): Los directorios fuente (`api/src` , `web/app` , `web/components` , `web/lib`) se montan de solo lectura. Los cambios toman efecto sin reconstruir contenedores. Solo los cambios a `package.json` , `Dockerfile` , `tsconfig.json` o nuevas dependencias requieren una reconstrucción.

Redes

Todos los servicios se unen a la red interna de Docker `noetia` . Solo Traefik tiene los puertos 80/443 expuestos al host. PostgreSQL, Redis, Meilisearch y Grafana no tienen enlaces de puertos al host — accesibles solo desde dentro de la red Docker o vía túnel SSH.

MinIO tiene su API accesible en `storage.noetia.app` (Traefik la enruta) para descargas con URLs prefirmadas. La consola de MinIO (puerto 9001) está expuesta solo en localhost, accesible vía túnel SSH.

4. Diseño de la API

Convenciones

- **REST con URLs basadas en recursos:** `GET /books/:id` , `POST /clubs` , `PATCH /users/me`
- **JWT en el header Authorization:** `Bearer <token>` en todos los endpoints protegidos
- **Validación vía DTOs con class-validator:** Todos los cuerpos de solicitud entrantes se validan a través de pipes NestJS antes de llegar a la lógica del servicio
- **Códigos de estado HTTP estrictamente seguidos:** 201 para recursos creados, 204 para eliminaciones, 400 para errores de validación, 401 para fallos de autenticación, 403 para fallos de autorización, 404 para recursos inexistentes

Grupos de Endpoints Principales

Grupo	Ruta Base	Endpoints Notables
Auth	<code>/auth</code>	POST register, login, logout, refresh, resend-confirmation, forgot-password, reset-password
Users	<code>/users</code>	GET/PATCH /me, DELETE /me (con confirmación ELIMINAR)
Books	<code>/books</code>	GET library, GET :id, POST progress, GET :id/sync-map, GET :id/stream
Stats	<code>/stats</code>	POST heartbeat, GET me
Subscriptions	<code>/subscriptions</code>	GET me, POST checkout, POST portal, POST webhook
Clubs	<code>/clubs</code>	CRUD completo + join, leave, ban, invite, polls, sessions, discussions
Social	<code>/social</code>	GET :platform/status, GET :platform/connect, POST :platform/publish

Sharing	/sharing	POST generate (imagen), GET :id
Library	/library	GET my-books, POST redeem, GET :bookId/access

Rate Limiting

Global: 120 solicitudes / 60 segundos por IP
Endpoints de Auth: 20 solicitudes / 60 segundos por IP
Endpoint de Métricas: solo interno (bloqueado a nivel Nginx/Traefik)
Webhooks de Stripe: exentos (ThrottlerModule @SkipThrottle)

5. Arquitectura de Seguridad

Seguridad de Autenticación

- Contraseñas hasheadas con bcrypt (factor de costo 12)
- Tokens de acceso: TTL de 15 minutos (vida corta para limitar el daño por robo de token)
- Tokens de refresh: almacenados en Redis con TTL de 7 días, rotados en cada uso
- Confirmación de email: tokens HMAC firmados de 24 horas, invalidados al usar
- Reseteo de contraseña: tokens HMAC firmados de 1 hora, invalidados al usar

Autorización

- `JwtAuthGuard` valida el token de acceso en cada ruta protegida
- `SubscriptionGuard` verifica el acceso al libro (estado de suscripción O registro de `book_purchase`) antes de servir `sync maps` o contenido
- Las acciones del club verifican membresía y rol (solo admin: ban, sesiones; moderador+: eliminar mensajes)
- Aislamiento de datos de usuario: cada consulta a la BD filtra por `userId` del sujeto JWT — no por ID de usuario basado en ruta

Seguridad de Infraestructura

- Firewall UFW: solo puertos 80, 443, 222 abiertos
- fail2ban: monitorea el puerto 222, máximo 5 reintentos, ban de 1 hora
- SSH en puerto 222 (cambiado del 22 predeterminado)
- Traefik TLS 1.2+ mínimo, certificados Let's Encrypt renovados automáticamente
- Buckets MinIO books/ y audio/: **privados** (se requieren URLs prefiradas)
- Bucket MinIO images/: **lectura pública** (las tarjetas de citas generadas son compartibles por diseño)
- Tokens OAuth sociales encriptados en reposo usando AES-256-CBC antes del almacenamiento en Redis

Protección de Contenido

- Texto del libro servido solo a través de endpoints autenticados con JWT y verificados por suscripción
- Audio servido mediante endpoint de streaming con range requests con los mismos guards
- Sync maps (timestamps de frases) con puerta `SubscriptionGuard` — evita extraer la estructura del libro sin acceso
- URLs prefiradas de MinIO vencen en 15 minutos

6. Monitoreo y Observabilidad

Métricas (Prometheus + Grafana)

- Tasa de solicitudes y latencia de API (p50, p95, p99) por endpoint
- Tasa de errores HTTP (4xx, 5xx) con alertas en picos sostenidos de 5xx
- Conteo de reinicios de contenedor (alertas en ≥ 3 reinicios en 15 minutos)
- Uso del pool de conexiones de PostgreSQL
- Uso de memoria de Redis

Alertas

- Alertas Grafana vía webhook → canal `#alerts` de Slack
- Alerta: tasa de 5xx de API > 5% por 3 minutos consecutivos
- Alerta: reinicios de contenedor > 3 en 15 minutos (período de gracia de 15 minutos al inicio)
- Alerta: uso de disco > 80%

Logging

- Logs de aplicación vía driver JSON de Docker, accesibles vía `docker compose logs -f`
- Logs de acceso de Traefik: todas las solicitudes logueadas con código de estado, latencia, upstream
- Sin agregación centralizada de logs a la escala actual (optimización de costos)

Backups

- **PostgreSQL**: Cron diario a las 02:00 UTC, `pg_dump` a archivo comprimido, retención de 7 días diaria + retención dominical de 4 semanas
- **MinIO**: Cron semanal a las 03:00 UTC, `mc mirror` a bucket de backup, rotación de 4 copias
- Ambos jobs de backup alertan ante fallo vía Grafana

7. Camino de Escalabilidad

Capacidad actual (VPS único, 8 vCPU, 24 GB): ~10,000 usuarios activos mensuales

Próximo hito de escalado (~10K-50K MAU):

1. Migrar PostgreSQL a RDS gestionado (Supabase o AWS RDS) — elimina la carga de mantenimiento de la BD
2. Actualizar VPS a 16 vCPU / 48 GB o separar api + worker en máquinas distintas
3. Añadir capa CDN delante de MinIO para entrega de imágenes (Cloudflare proxying `storage.noetia.app`)
4. Añadir Redis Cluster o Redis gestionado (Upstash) para escalar sesiones y colas

Arquitectura futura (~100K+ MAU):

1. Kubernetes (K3s o EKS/GKE gestionado) para escala horizontal de api y web
2. Réplicas de lectura separadas para PostgreSQL (consultas de analytics y stats)
3. Meilisearch Cloud (eliminar mantenimiento de búsqueda self-hosted)
4. Cluster de generación de imágenes dedicado (acelerado por GPU para portadas generadas por IA en el futuro)

Documento mantenido por el Desarrollador Backend y el Ingeniero DevOps. Última actualización: 25 de mayo de 2026.